BRIEFR System Design

Version: 1.1 (beta)
Last updated: 2026-06-08
Source of truth: `/workspace` codebase — see `Beta V1.2.md` for near-future roadmap

---

1. Overview

BRIEFR is a CVE intelligence platform that ingests vulnerability data from NVD, CISA KEV, EPSS, and MITRE sources into a local SQLite database, enriches records with threat-context feeds (OTX, Sploitus, GreyNoise, OSV, CIRCL), and presents them through a React analyst UI with IOC lookup, risk scoring, correlation, and PDF export.

It is built for security analysts, small security teams, and solo researchers who need a single-pane view of what is exploitable, what is in KEV, and what matches their stack — without standing up a full SIEM or commercial threat-intel platform.

The core problem it solves is **analyst time**: aggregating scattered CVE metadata, exploitation signals, ATT&CK mapping, and IOC enrichment into one fast, dark-mode workflow that runs on a single server with optional API keys.

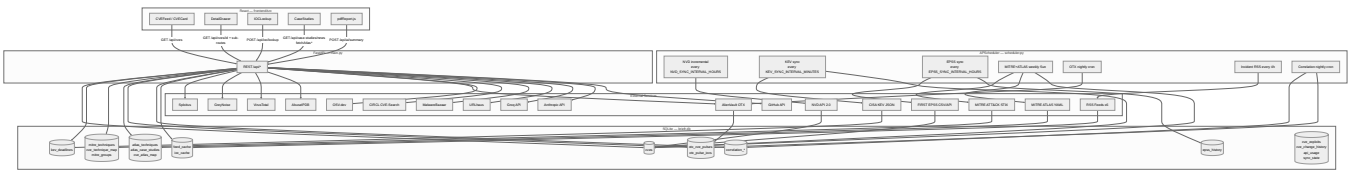
---

2. Architecture

Four-layer model

```
Feed Ingestion  →  SQLite DB  →  FastAPI API  →  React UI
(scheduler.py)     (database.py)   (main.py)      (frontend/src)
```

Architecture diagram



DB tables → primary API readers

Table(s)	Primary endpoints	Frontend consumers
`cves`	`GET /api/cves`, `GET /api/cves/{id}`, `GET /api/stats`	CVEFeed, CVECard, DetailDrawer, StatsRow, TimelineHeatmap
`kev_deadlines`	`GET /api/kev/deadlines`, embedded in CVE detail	Sidebar, DetailDrawer sentences
`epss_history`	`GET /api/cves/{id}/epss-history`, momentum	DetailDrawer EPSS sparkline
`mitre_techniques`, `cve_technique_map`	`GET /api/techniques/top`, CVE `techniques` field	Sidebar, DetailDrawer Intel tab
`atlas_*`, `cve_atlas_map`	`GET /api/atlas/*`, `GET /api/cves/{id}` (per-CVE fields)	DrawerAtlasSection, CaseStudies (global list)
`otx_*`	CVE detail, correlation, IOC lookup	DetailDrawer Intel tab, IOCLookup
`feed_cache`, `ioc_cache`	Internal — speeds enrichment	Transparent to UI
`correlation_*`	`GET /api/cves/{id}/correlation`	DetailDrawer correlation section
`cve_exploits`	Via Sploitus loader in CVE detail	DetailDrawer Intel tab
`cve_change_history`	`GET /api/changes`	— (API only)
`api_usage`	`GET /api/usage`, `GET /api/usage/ioc`	IOCLookup quota display

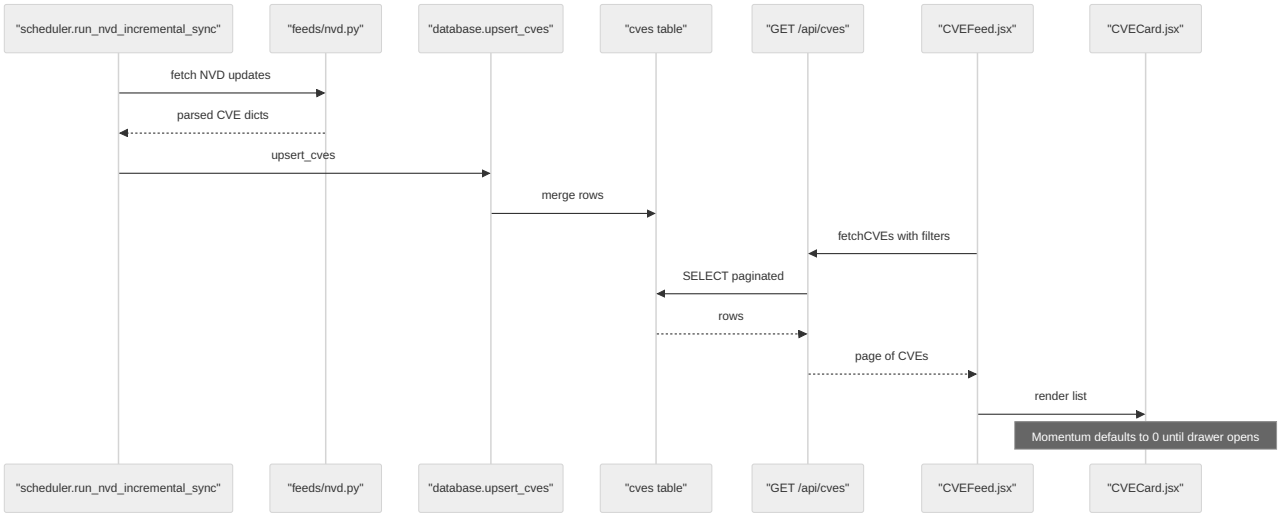
---

3. Data Flow

A. CVE lifecycle

1. **Ingest:** `scheduler.run_nvd_incremental_sync` → `feeds/nvd.py:fetch_nvd_cve_updates` (NVD REST 2.0, watermark in `sync_state`).
2. **Persist:** `database.upsert_cves` → `cves` table (`ON CONFLICT DO UPDATE`), optional `cve_change_history` rows.
3. **Post-process:** strip auto-summaries, backfill display fields, `enrich_cves_extended` (Sploitus/CIRCL).
4. **List:** `GET /api/cves` builds SQL from `_build_cve_filters`, paginates (`page`, `limit` max **50**).
5. **UI:** `CVEFeed.jsx:loadPage` → `fetchCVEs` → `CVECard.jsx` renders each row.

flow cve feed

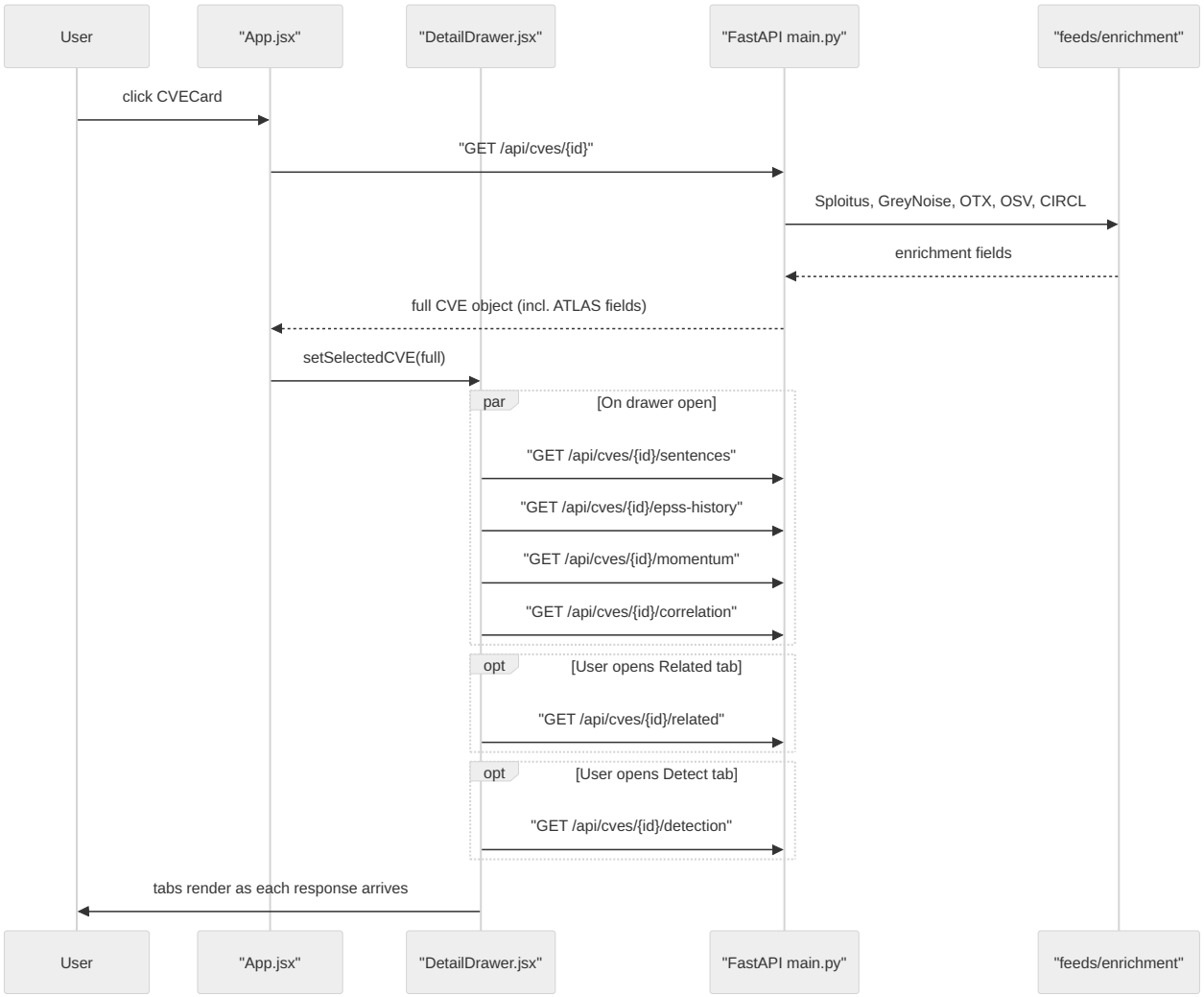


B. CVE detail drill-down

1. **Card click:** `App.jsx:handleSelectCVE` sets list CVE, then `fetchCVE(cve_id)` → `GET /api/cves/{id}`.
2. **Server enrichment (serial awaits in handler):** Sploitus exploits, GreyNoise scans, OTX pulses, OSV packages, CIRCL merge (`main.py:912–971`).
3. **Drawer opens** with enriched CVE; parallel client fetches on `cve_id` change:
   - `GET /api/cves/{id}/sentences` (immediate)
   - `GET /api/cves/{id}/epss-history` (immediate)
   - `GET /api/cves/{id}/momentum` (immediate)
   - `GET /api/cves/{id}/correlation?sector=` (immediate)
4. **Lazy tab fetches:**
   - `GET /api/cves/{id}/related` — only when **Related** tab active
   - `GET /api/cves/{id}/detection` — only when **Detect** tab first opened
5. **OTX pulse IOCs:** loaded via CVE detail `otx_pulses`; pulse IOC drill-down uses `GET /api/otx/pulses/{id}/iocs`.

ATLAS wiring: `GET /api/cves/{id}` returns `has_ai_context`, `atlas_techniques`, and `atlas_case_studies` via `database.get_atlas_techniques_for_cve` / `get_atlas_case_studies_for_cve` for `DrawerAtlasSection.jsx`.

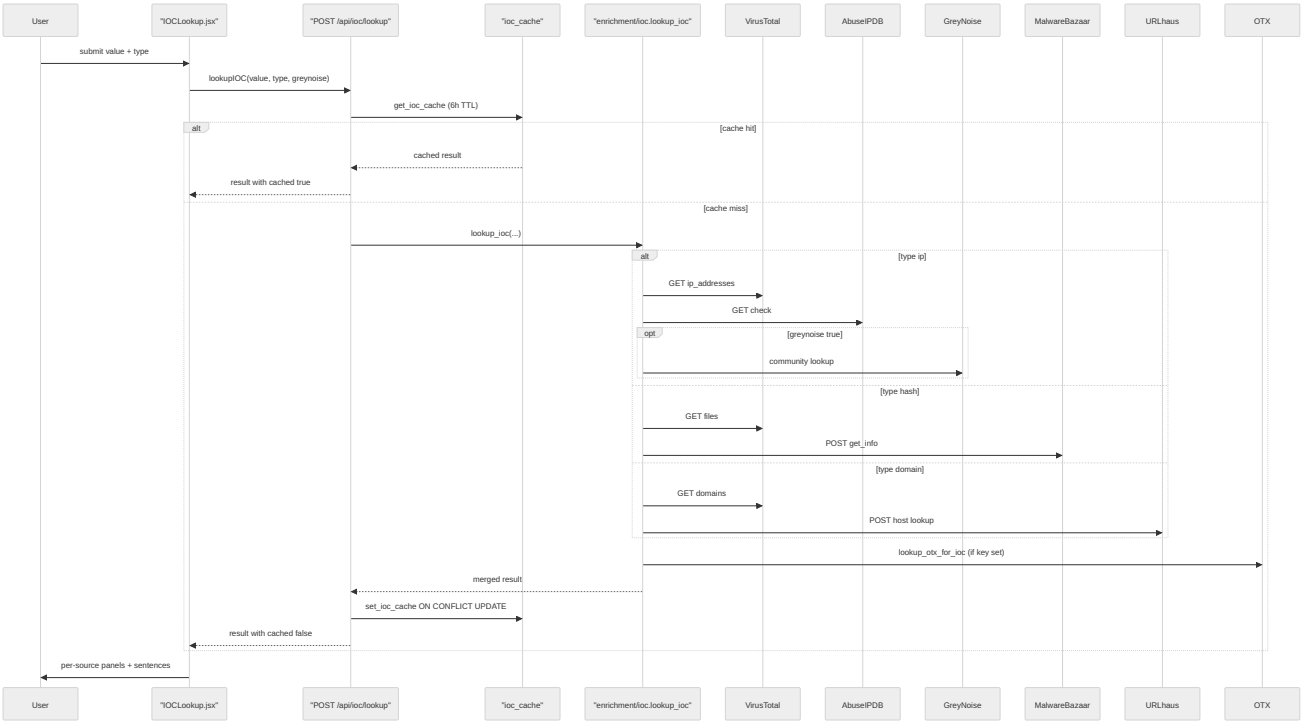
flow cve detail



C. IOC lookup

1. **Input:** `IOCLookup.jsx` validates type (`ip` | `hash` | `domain`), optional GreyNoise opt-in.
2. **API:** `POST /api/ioc/lookup` → `get_ioc_cache` (6h) or `enrichment/ioc.lookup_ioc` .
3. **Per-type enrichment (sequential within shared httpx client, not asyncio.gather):**
   - **IP:** VirusTotal → AbuseIPDB → (optional) GreyNoise → OTX
   - **Hash:** VirusTotal → MalwareBazaar
   - **Domain:** VirusTotal → URLhaus → OTX
4. **Cache write:** `set_ioc_cache` with `ON CONFLICT DO UPDATE` .
5. **UI:** per-source result cards and template sentences from `templates/intelligence.py` .

flow ioc lookup



D. Risk scoring (v1.1b)

Client-side (`frontend/src/scoring/riskScore.js:calculateRiskScore`):

Component	Weight
Asset profile match	0.35
KEV status	0.25
EPSS	0.15
Exploit availability	0.10
CVSS	0.10
Momentum	0.05

Momentum fetched lazily from `GET /api/cves/{id}/momentum` → `scoring/risk.py:calculate_momentum` (EPSS trend, OTX pulse recency, recent KEV, rapid exploitation signals). Cached in `momentumCache.js` for card arrows.

Display: `DetailDrawer.jsx` Overview tab — `RiskScoreBreakdown` (not Correlation tab). Cards use momentum `0` until drawer fetch updates cache.

Duplication debt: same weights/logic mirrored in `backend/scoring/risk.py` (server momentum only today).

E. Incidents & News feed

1. **UI:** `CaseStudies.jsx` calls `loadCaseStudyFeed()` → `GET /api/case-studies/feed?atlas_limit=80`.
2. **Client cache:** `caseStudyFeed.js` holds a 5-minute session cache to avoid refetching on tab switches.
3. **Server:** `case_study_feed.py:fetch_combined_case_study_feed` opens **one** SQLite connection, then sequentially:
   - `fetch_all_incident_news(db)` — 6 RSS sources via `incident_news.py` (30 min `feed_cache` per source)
   - `_load_atlas_cards(db)` — ATLAS case studies from `atlas_case_studies` table
4. **Merge:** Cards sorted by `publishedAt` descending; per-source errors collected in `errors[]` without failing the whole feed.
5. **Editorial filter:** `incident_news.py` excludes non-security RSS items by title pattern (e.g. Dark Reading **"Name That Toon"** contest). Filter applies on parse and when serving cached rows; malformed cache entries are skipped defensively.
6. **Scheduler:** `run_incident_news_refresh` pre-warms RSS caches every 4 hours (`scheduler.py`).

startup